

Abstraction in Python

From repeated mistakes to reliable behavioral contracts

The promise of this chapter

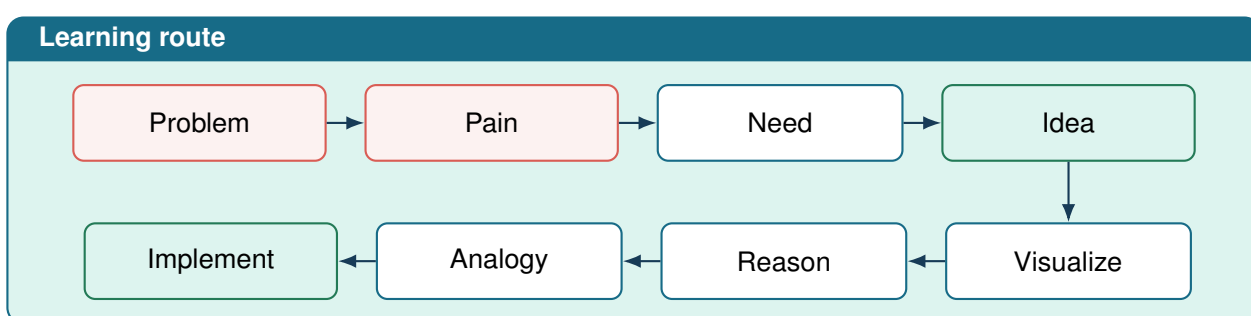
You will not memorize “an abstract class is a class that cannot be instantiated.” You will discover *why* teams need enforceable contracts, then see how Python’s `abc` module expresses them. By the end, you will be able to design, explain, debug, and interview confidently about abstraction.

Contents

1	The Road to Abstraction	1
1.1	Readiness: the ideas already in your toolbox	1
1.2	The recurring problem: promises that code does not enforce	2
1.3	Why ordinary inheritance is not enough	2
1.3.1	Learning Lab 1: the forgotten safety step	2
1.4	Discovering the contract	4
2	The Idea Gets a Name	5
2.1	What abstraction means—now that we need it	5
2.1.1	Abstraction is not simply information hiding	5
2.2	Analogies that preserve the contract idea	6
2.3	Why Python provides <code>abc</code>	6
3	Building the First Abstract Class	7
3.1	Learning Lab 2: transform the payment design	7
3.1.1	Step 1 and 2: normal parent, visible failure	7
3.1.2	Step 3 and 4: express the contract	7
3.2	Attempting to create the incomplete idea	9
4	Abstract Methods in Depth	10
4.1	What an abstract method really says	10
4.1.1	Can abstract methods contain code? Yes	10
4.2	What may an abstract class contain?	11
4.2.1	Learning Lab 3: one class demonstrating the possibilities	11
5	Contracts with Several Obligations	14
5.1	Learning Lab 4: production-shaped payment workflow	14
5.2	Cloud storage contract	15
5.3	Notification contract	16
6	Abstraction in Real Projects	17
6.1	Architecture: stable core, replaceable edges	17

6.2	Seven project blueprints	18
6.3	Complete mini-project: authentication framework	18
6.4	When abstraction improves maintainability	20
6.5	When not to use it	20
7	Separate but Connected Ideas	21
7.1	Abstraction versus encapsulation	21
7.2	Abstraction versus polymorphism	21
7.3	Python ABCs, Java-style interfaces, protocols, and duck typing	22
8	Forty-Five Beginner Traps	23
9	Interview Thinking	26
9.1	Questions with reasoning-first answers	26
9.2	A strong 60-second explanation	27
10	Practice Studio	28
10.1	Easy: build the mental model	28
10.2	Medium: implementation	28
10.3	Hard: design	28
10.4	Debugging challenges	29
10.5	Output prediction	29
10.6	Fill in the blanks	30
10.7	Interview coding tasks	30
10.8	Selected answers	30
11	Master Revision Map	31
11.1	Cheat sheet	31
11.2	Mind map	31
11.3	Flow chart: from pain to working object	32
11.4	Decision tree: should I create an abstraction?	32
11.5	Final revision notes	32

The Road to Abstraction



1.1 Readiness: the ideas already in your toolbox

Abstraction is not an isolated trick. It connects five ideas you already know.

Idea	Quick revision	Why abstraction needs it
Class	A blueprint that groups data and behavior.	An abstract class is a special blueprint: it may contain shared behavior and required behavior.
Object	A runtime instance created from a class.	Some blueprints are intentionally incomplete, so creating their objects would be dishonest.
Inheritance	A child class receives or specializes a parent.	The parent publishes the contract; children inherit the obligation.
Overriding	A child replaces an inherited method.	Each concrete child supplies its own way of performing a required operation.
Polymorphism	One common call can trigger different implementations.	Once every child honors the contract, client code safely makes the same call on many objects.

Pause and Predict

Before continuing, answer aloud:

1. If two child classes override `send()`, why can one function call `item.send()` on either object?
2. Does inheritance force a child to replace an ordinary inherited method?
3. Should an object be created from a blueprint that cannot perform its promised job?

If question 2 feels uncomfortable, excellent—that discomfort is the doorway to abstraction.

1.2 The recurring problem: promises that code does not enforce

Imagine a payment platform. Today it supports cards and wallets; next month another developer adds bank transfers. Every payment must authenticate, make the payment, and issue a receipt.

Problem → Pain → Need

The team writes these rules in documentation. A new developer remembers `pay()` but forgets `authenticate()`. Tests do not cover the new class. The program still creates the object. Money-moving code now exists without the safety step.

Need: make an incomplete payment class fail immediately, before it enters a transaction.

This same shape appears everywhere:

- every vehicle must start and stop;
- every notification must validate and send;
- every shape must calculate area;
- every animal in a simulation must move;
- every employee type must calculate pay;
- every storage provider must upload and download;
- every login provider must authenticate;
- every report exporter must export.

The question is not “Can a developer write these methods?” The question is “How can the design *guarantee* that every future implementation supplies them?”

1.3 Why ordinary inheritance is not enough

1.3.1 Learning Lab 1: the forgotten safety step

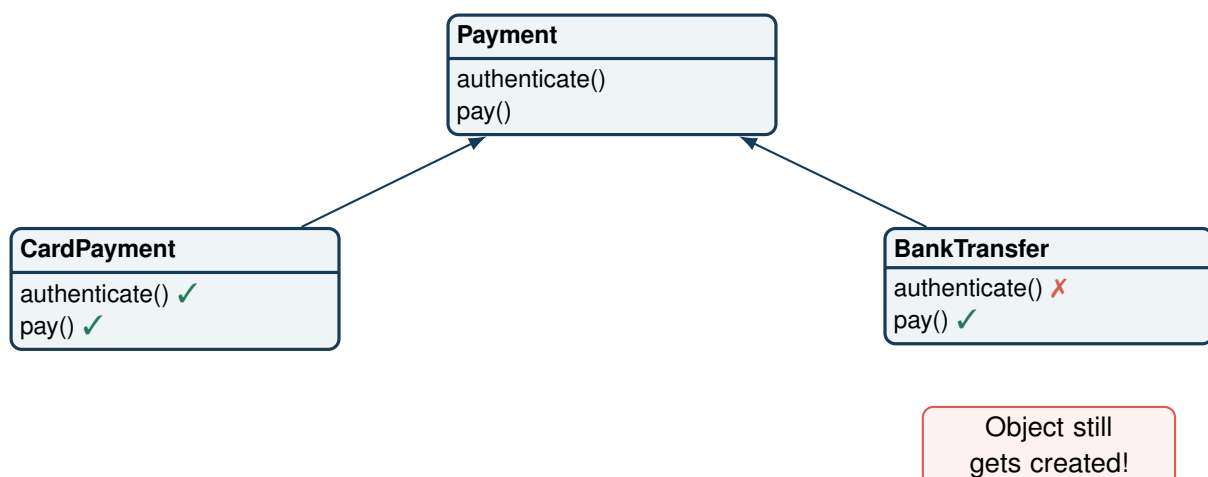
Problem → Pain → Need

Problem. All payment types should authenticate before paying.

Existing approach. Put placeholder methods in a normal parent.

Why insufficient. A child silently inherits the placeholder. Python sees a valid method name, even though the promised behavior is missing.

Visualization



Pseudo code

```

DEFINE ordinary Payment parent
    authenticate → do nothing useful
    pay → do nothing useful
DEFINE BankTransfer child
    pay → print success
CREATE BankTransfer
CALL authenticate, then pay

```

Python code and driver code

```

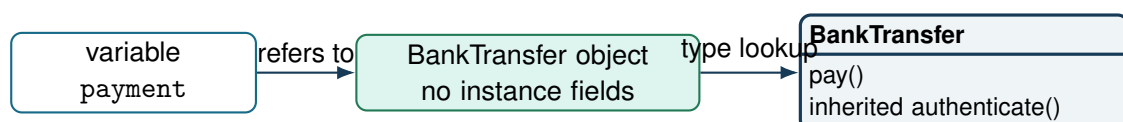
1 class Payment:
2     def authenticate(self):
3         pass                                     # dangerous placeholder
4
5     def pay(self, amount):
6         pass
7
8
9 class CardPayment(Payment):
10    def authenticate(self):
11        print("Card verified")
12
13    def pay(self, amount):
14        print(f"Card paid ${amount}")
15
16
17 class BankTransfer(Payment):
18     # authenticate() was forgotten
19     def pay(self, amount):
20         print(f"Bank transferred ${amount}")
21
22
23 # Driver code
24 payment = BankTransfer()
25 payment.authenticate()
26 payment.pay(500)

```

Expected console output

```
Bank transferred $500
```

Output, line by line. There is only one line. The inherited `authenticate()` executes `pass`, so it produces no evidence that authentication occurred. Then `pay()` announces success. Silence was mistaken for safety.

Memory and object diagram**Dry run**

1. Python creates a `BankTransfer`; ordinary inheritance imposes no completeness check.
2. Lookup for `authenticate` fails in `BankTransfer`, then succeeds in `Payment`.
3. The inherited `pass` returns `None`; execution continues.

4. Lookup for `pay` finds the child implementation, which prints success.

Complexity. Method lookup and each shown operation are effectively $O(1)$ here. The important issue is correctness, not speed.

▲ Warning

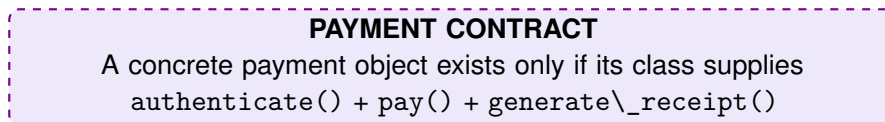
Replacing `pass` with `raise NotImplementedError` is louder, but failure occurs only when the forgotten method is called. We want failure at object creation, closer to the design mistake.

Interview Thinking

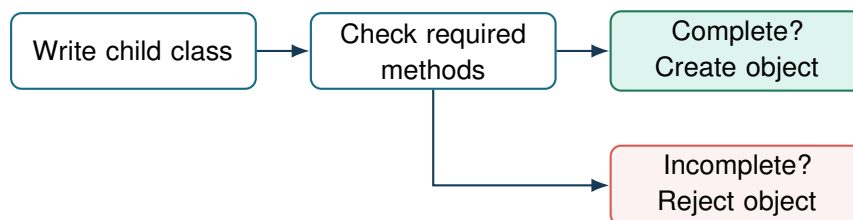
Why is inheritance alone insufficient? Ordinary inheritance reuses behavior and permits overriding; it does not require that selected methods be overridden. A contract needs explicit enforcement.

1.4 Discovering the contract

Do not think about syntax yet. Imagine that `Payment` could attach this message to itself:



Now the sequence changes:



A **contract** is a set of behavioral promises a concrete subtype must fulfill. It says *what capabilities must exist*. It deliberately leaves each subtype free to decide *how* to perform them.

Pause and Predict

Would the contract say that a card and a bank transfer must use the same authentication algorithm? No. That would remove useful variation. It says only that both must offer a real authentication operation.

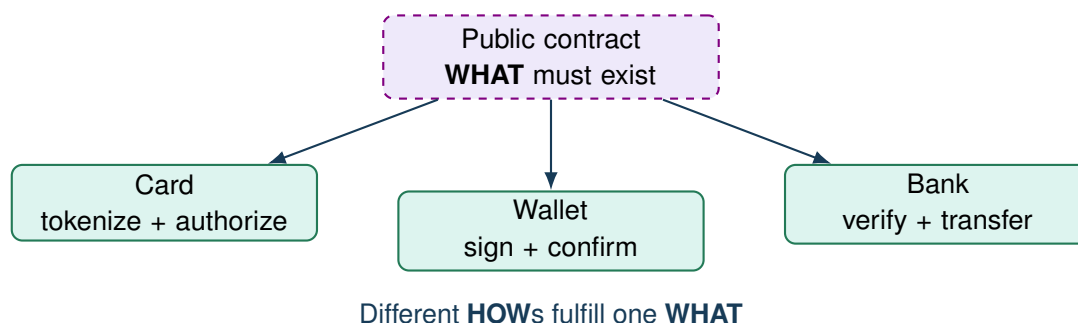
The Idea Gets a Name

2.1 What abstraction means—now that we need it

Working definition

Abstraction in Python is a way to define a contract that every concrete subclass must follow while allowing each subclass to provide its own implementation.

Many books say abstraction means “showing essential features while hiding implementation details.” That statement is useful but incomplete for beginners. It describes the *view from the caller*: call `pay()` without knowing gateway internals. Python abstract base classes add an important *designer’s view*: declare required behavior and prevent incomplete implementations from becoming concrete objects.



2.1.1 Abstraction is not simply information hiding

Question	Abstraction	Information hiding / encapsulation
Main concern	Which behavior is essential?	Which internal state or details should be controlled?
Typical tool	ABC, abstract method, protocol, public API	naming conventions, properties, carefully chosen methods
Example	Every storage provider must implement <code>upload</code> .	A storage token is kept in <code>_token</code> and changed through validated code.
Can occur alone?	Yes; a contract may expose no private data.	Yes; a simple class may protect state without an abstract parent.

2.2 Analogies that preserve the contract idea

Analogy	Visible contract (WHAT)	Hidden or variable implementation (HOW)	Python connection
Restaurant menu	“Order soup” is available.	Recipe, stove, timing, and chef vary.	Method name and result are stable; subclass algorithm varies.
Remote control	Power and volume buttons exist.	TV brands interpret signals differently.	Common client calls work across concrete types.
Car dashboard	Start, brake, and speed controls exist.	Electric and combustion systems differ.	Vehicle contract, specialized implementations.
Electric plug	Devices must fit voltage/pin rules.	A lamp and laptop use electricity differently.	Compatibility comes from honoring an agreed shape.
Mobile charger	Supplies a compatible charging service.	Power negotiation and battery chemistry vary.	Contract separates caller from mechanism.
Payment gateway	Authenticate, pay, receipt.	Card network, wallet, or bank workflow.	Abstract methods mark required operations.
ATM	Withdraw and check balance.	Banks and ledgers differ.	Interface remains stable across providers.
Boarding pass	Identifies passenger, flight, and gate.	Airline reservation systems differ.	Essential data/behavior is standardized.

★ Professional Tip

An analogy is accurate only if it preserves the rule: a stable promise is visible, multiple implementations can fulfill it, and users depend on the promise rather than the mechanism.

2.3 Why Python provides `abc`

Large programs needed a standard way to describe and test behavioral families more formally than comments or conventions. Python’s standard-library `abc` module means *Abstract Base Classes*. It provides:

ABC a convenient parent class whose metaclass tracks abstract obligations;

`abstractmethod`

a decorator that marks a method as required before a subclass can be concrete.

Historically, Python has always valued dynamic “if it behaves correctly, use it” style. ABCs added an optional, explicit contract mechanism for frameworks and APIs where early enforcement and shared categorization are valuable. They complement duck typing; they do not replace it.

Building the First Abstract Class

3.1 Learning Lab 2: transform the payment design

Problem → Pain → Need

Problem. The ordinary parent allowed an incomplete child.

Need. State required operations in the parent and reject incomplete concrete objects.

Idea. Make Payment an abstract base class and mark its obligations.

3.1.1 Step 1 and 2: normal parent, visible failure

The previous lab used placeholders. Its essential flaw can be summarized as:

```
class Payment:
    def authenticate(self):
        pass # hope is not enforcement
```

3.1.2 Step 3 and 4: express the contract

Pseudo code

```
IMPORT contract tools
DEFINE Payment AS an abstract base class
    REQUIRE authenticate
    REQUIRE pay
    PROVIDE concrete helper show_provider
DEFINE CardPayment FROM Payment
    IMPLEMENT authenticate
    IMPLEMENT pay
```

Python implementation

```
1 from abc import ABC, abstractmethod
2
3
4 class Payment(ABC):
5     def __init__(self, provider):
6         self.provider = provider
```

```

7
8     @abstractmethod
9     def authenticate(self):
10         """Verify that the payer is allowed to continue."""
11
12     @abstractmethod
13     def pay(self, amount):
14         """Transfer the requested amount."""
15
16     def show_provider(self):                # concrete method
17         print(f"Provider: {self.provider}")
18
19
20 class CardPayment(Payment):
21     def authenticate(self):
22         print("Card verified")
23
24     def pay(self, amount):
25         print(f"Card paid ${amount}")
26
27
28 # Driver code
29 payment = CardPayment("NorthStar Card Network")
30 payment.show_provider()
31 payment.authenticate()
32 payment.pay(500)

```

Expected output

```

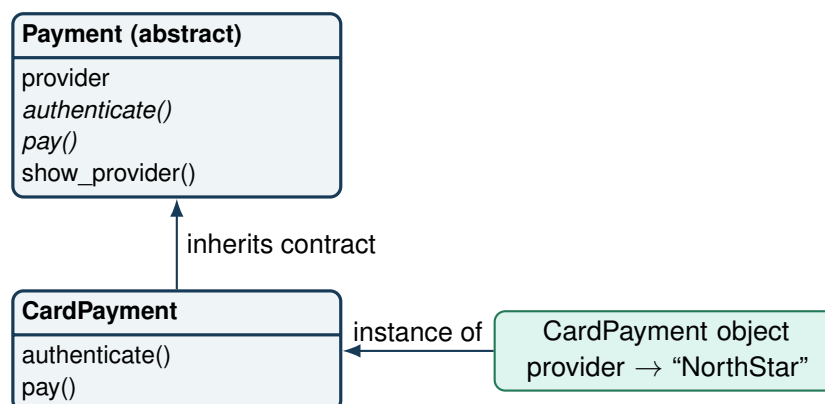
Provider: NorthStar Card Network
Card verified
Card paid $500

```

Output explanation

1. The constructor stores the inherited provider; the concrete helper prints it.
2. Dynamic dispatch selects `CardPayment.authenticate`.
3. The same object receives `pay(500)`, selecting the card-specific implementation.

Class relationship and object memory



Dry run

1. Class creation records two abstract method names on `Payment`.
2. Class creation sees that `CardPayment` supplies both names; its outstanding set is empty.

3. Object creation is therefore allowed; inherited `Payment.__init__` runs.
4. Calls use the shared method where appropriate and overridden methods where behavior varies.

Line-by-line logic

Lines	Meaning
1	Import the standard contract-building tools.
4	Inheriting from ABC opts this hierarchy into abstract-method enforcement.
5–6	An abstract class may initialize shared state.
8–15	Each decorator marks the following method as a required promise. The docstrings explain intent.
17–18	A normal method is inherited and usable by concrete children.
21–26	The child fulfills both promises using matching callable names.
30–33	The driver creates only a complete child and uses shared plus specialized behavior.

Common mistakes and edge cases

- The decorator must be written directly above the method it marks.
- A matching name is required, but Python does not fully enforce parameter type annotations or return types.
- Misspelling `authenticate` creates a new method and leaves the obligation unfulfilled.
- A child may remain abstract intentionally; only object creation is blocked.

Interview Thinking

What exactly is enforced? Python prevents instantiation while the class has any unimplemented abstract methods. It does not prove business correctness: an implementation could still be empty or wrong. Contracts reduce one class of mistake; tests and reviews remain necessary.

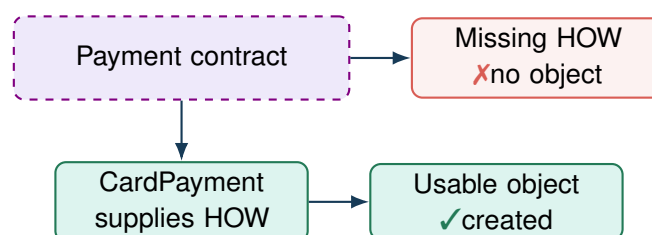
3.2 Attempting to create the incomplete idea

```
1 # Driver code
2 payment = Payment("Impossible Provider")
```

Typical output (wording can vary slightly by Python version):

```
TypeError: Can't instantiate abstract class Payment
with abstract methods authenticate, pay
```

Python refuses because `Payment` describes the category but cannot honor its own complete promise. An object represents something usable now; an abstract class represents something that still needs specialization.



Abstract Methods in Depth

4.1 What an abstract method really says

An abstract method says: “This operation belongs to the public meaning of this family, but this class is not the final authority on how it is done.” It does *not* mean the method must contain no code.

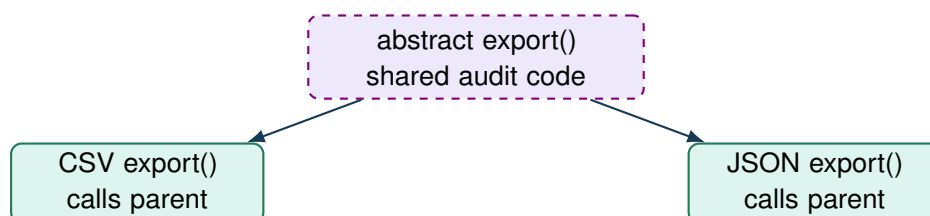
4.1.1 Can abstract methods contain code? Yes

Problem → Pain → Need

Problem. Every exporter must perform a shared audit step, yet each must format data differently.

Need. Provide useful default work while still requiring an explicit child decision.

Visualization and pseudo code



REQUIRE every exporter to override export
 PLACE shared audit behavior in the abstract method
 CHILD calls the parent portion, then formats its data

Python code, driver, and output

```

1 from abc import ABC, abstractmethod
2
3 class Exporter(ABC):
4     @abstractmethod
5     def export(self, records):
6         print(f"Audit: exporting {len(records)} record(s)")
7
8 class CSVExporter(Exporter):
9     def export(self, records):
10        super().export(records)
  
```

```

11         print("CSV: " + ",".join(records))
12
13     exporter = CSVExporter()                # driver
14     exporter.export(["Ada", "Linus"])

```

```

Audit: exporting 2 record(s)
CSV: Ada,Linus

```

Output and dry run. The child override makes the class concrete. Its first line explicitly calls the parent implementation, producing the audit line. It then performs CSV-specific formatting. For n records totaling m characters, joining is $O(m)$ and the list holds $O(n)$ references; the contract mechanism adds no meaningful algorithmic cost.

▲ Warning

Parent abstract-method code is not automatically executed. A child must call it, usually with `super()`. Use this pattern only when the shared portion is genuinely universal; otherwise prefer a separate concrete helper method.

Interview Thinking

Abstract means “override is still required,” not “body must be empty.” Python permits a body, docstring, return value, or shared algorithm in an abstract method.

4.2 What may an abstract class contain?

Member	Allowed?	Reason / example
Constructor	✓Yes	Initialize state shared by concrete subclasses; children may call it through <code>super().__init__()</code> .
Concrete methods	✓Yes	Reuse stable behavior such as validation or logging.
Instance variables	✓Yes	Set them in the constructor, e.g. <code>self.name</code> .
Class variables	✓Yes	Store shared constants such as <code>currency = "USD"</code> .
Properties	✓Yes	A property may be concrete or abstract.
Static methods	✓Yes	Combine <code>@staticmethod</code> with <code>@abstractmethod</code> when subclasses must supply a utility.
Class methods	✓Yes	Combine <code>@classmethod</code> with <code>@abstractmethod</code> for required alternative constructors.
“Private” methods	✓Yes	Double underscore triggers name mangling; use sparingly in extensible parents.
Protected members	✓Convention	A single underscore communicates “for internal/subclass use” but is not access enforcement.
Abstract methods	✓Yes	They are the explicit obligations.

4.2.1 Learning Lab 3: one class demonstrating the possibilities

```

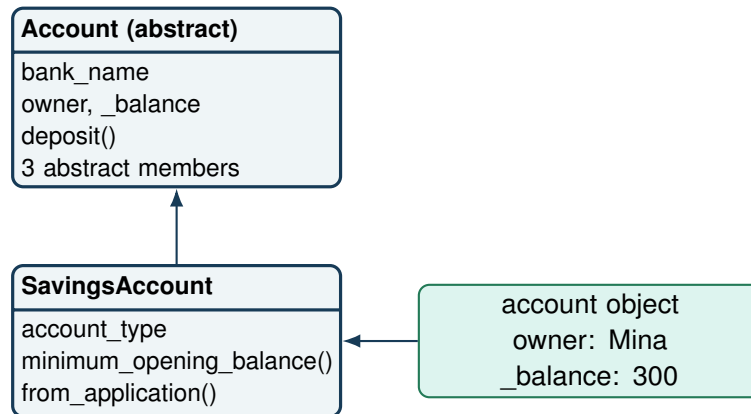
1  from abc import ABC, abstractmethod
2
3  class Account(ABC):
4      bank_name = "Prism Bank"                # class variable
5
6      def __init__(self, owner, balance=0):    # constructor
7          self.owner = owner                  # public instance variable
8          self._balance = balance             # protected convention
9          self.__audit_code = "A7"           # name-mangled member
10
11     @property
12     def balance(self):                       # concrete property
13         return self._balance
14
15     @property
16     @abstractmethod
17     def account_type(self):                  # abstract property
18         ...
19
20     @staticmethod
21     @abstractmethod
22     def minimum_opening_balance():           # abstract static method
23         ...
24
25     @classmethod
26     @abstractmethod
27     def from_application(cls, form):          # abstract class method
28         ...
29
30     def deposit(self, amount):               # concrete method
31         if amount <= 0:
32             raise ValueError("amount must be positive")
33         self._balance += amount
34         self.__record_audit()
35
36     def __record_audit(self):                # name-mangled helper
37         print("Audit recorded")
38
39
40 class SavingsAccount(Account):
41     @property
42     def account_type(self):
43         return "savings"
44
45     @staticmethod
46     def minimum_opening_balance():
47         return 100
48
49     @classmethod
50     def from_application(cls, form):
51         return cls(form["owner"], form["deposit"])
52
53
54 # Driver code
55 account = SavingsAccount.from_application(
56     {"owner": "Mina", "deposit": 250}
57 )
58 account.deposit(50)
59 print(account.account_type, account.balance)

```



```
Audit recorded  
savings 300
```

Object diagram and dry run



The class method receives `SavingsAccount` as `cls`, constructs it, and the inherited constructor stores state. `deposit` validates, updates $250 + 50$, and calls the mangled audit helper. Finally the concrete property and balance print. Each operation is $O(1)$.

Debugging Lens

Decorator order matters: for properties, static methods, and class methods, place `@abstractmethod` closest to `def`. Reversing the order can raise an error because the outer descriptor may not allow its abstract flag to be changed.

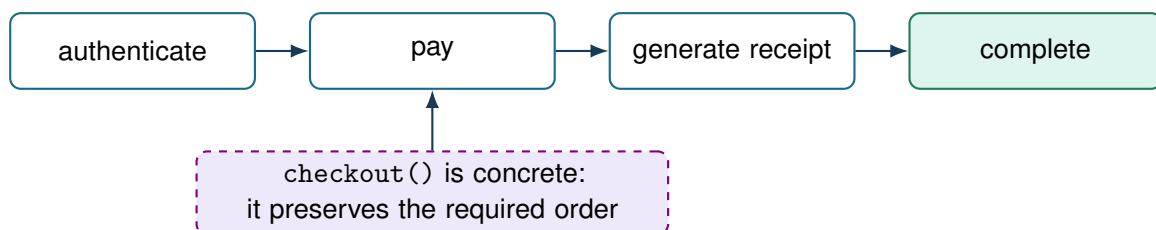
Contracts with Several Obligations

5.1 Learning Lab 4: production-shaped payment workflow

Problem → Pain → Need

A payment is not one action. If receipt generation is optional by accident, accounting becomes inconsistent. The contract must require all three stages while a concrete workflow controls their order.

Visualization



Pseudo code and Python code

CONTRACT requires authenticate, pay, generate_receipt
 TEMPLATE METHOD checkout calls them in safe order
 EACH gateway supplies the three variable steps

```

1 from abc import ABC, abstractmethod
2
3 class PaymentGateway(ABC):
4     @abstractmethod
5     def authenticate(self): ...
6
7     @abstractmethod
8     def pay(self, amount): ...
9
10    @abstractmethod
11    def generate_receipt(self): ...
12
13    def checkout(self, amount):
14        self.authenticate()
15        self.pay(amount)
16        self.generate_receipt()
  
```

```

17
18 class WalletGateway(PaymentGateway):
19     def authenticate(self): print("Wallet PIN accepted")
20     def pay(self, amount): print(f"Wallet paid ${amount}")
21     def generate_receipt(self): print("Receipt W-104 created")
22
23 # Driver code
24 gateway = WalletGateway()
25 gateway.checkout(80)

```

```

Wallet PIN accepted
Wallet paid $80
Receipt W-104 created

```

Output dry run. The concrete template method calls three operations on the same object. Polymorphic lookup selects the wallet implementation each time. Output order proves the safety sequence. Each shown step is $O(1)$, excluding real network/database costs.

Mistakes and edge cases. An exception in authentication should stop later stages; real money code must also handle idempotency, timeouts, rollback/compensation, secure secrets, and durable receipt storage. Abstraction organizes responsibilities but does not supply those policies.

5.2 Cloud storage contract

Problem → Pain → Need

An application should switch between local, S3-like, or laboratory storage without rewriting the research workflow. Requiring upload, download, and delete makes the dependency explicit.

```

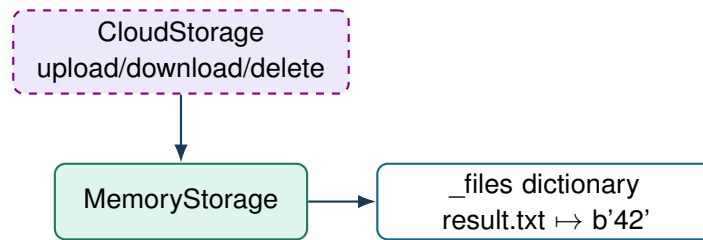
1 from abc import ABC, abstractmethod
2
3 class CloudStorage(ABC):
4     @abstractmethod
5     def upload(self, name, data): ...
6     @abstractmethod
7     def download(self, name): ...
8     @abstractmethod
9     def delete(self, name): ...
10
11 class MemoryStorage(CloudStorage):
12     def __init__(self): self._files = {}
13     def upload(self, name, data): self._files[name] = data
14     def download(self, name): return self._files[name]
15     def delete(self, name): del self._files[name]
16
17 # Driver code
18 store = MemoryStorage()
19 store.upload("result.txt", b"42")
20 print(store.download("result.txt"))
21 store.delete("result.txt")
22 print("Deleted")

```

```

b'42'
Deleted

```

Relationship, memory, and dry run.

Upload inserts a dictionary entry; download retrieves it; delete removes it. Average-case dictionary access is $O(1)$, with $O(m)$ memory for stored bytes. Missing names raise `KeyError`; a production contract should document whether to translate that error.

5.3 Notification contract

```

1 from abc import ABC, abstractmethod
2
3 class Notification(ABC):
4     @abstractmethod
5     def validate(self, recipient): ...
6     @abstractmethod
7     def send(self, recipient, message): ...
8     @abstractmethod
9     def log(self, recipient): ...
10
11 class EmailNotification(Notification):
12     def validate(self, recipient): return "@" in recipient
13     def send(self, recipient, message):
14         if not self.validate(recipient):
15             raise ValueError("invalid email")
16         print(f"Email to {recipient}: {message}")
17     def log(self, recipient): print(f"Logged: {recipient}")
18
19 # Driver code
20 notice = EmailNotification()
21 notice.send("learner@example.org", "Welcome")
22 notice.log("learner@example.org")

```

```

Email to learner@example.org: Welcome
Logged: learner@example.org

```

Dry run and edge case. Validation returns true because the address contains @; send prints, then log records. The simplified validation is $O(n)$ and is not RFC-complete. Real systems handle provider failures, retries, duplicate delivery, privacy, and structured logging.

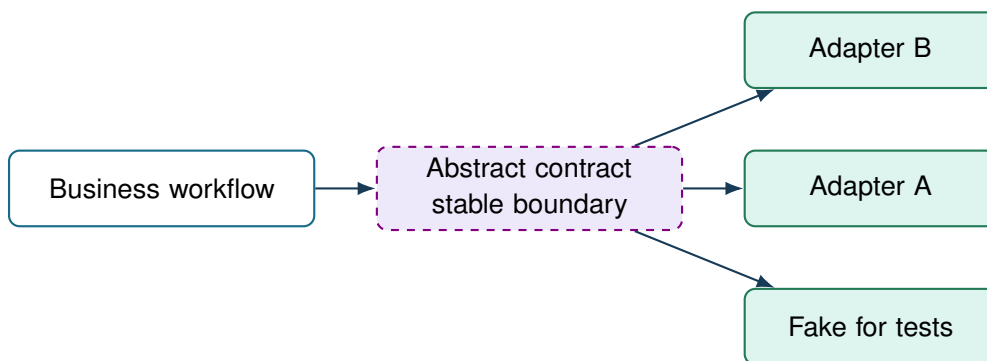
Interview Thinking

Three methods are not automatically better than one. Choose obligations that every member of the family can honestly support. A forced `delete()` on immutable archival storage might reveal a badly chosen contract; consider smaller capability-based ABCs.

Abstraction in Real Projects

6.1 Architecture: stable core, replaceable edges

Abstraction is most valuable at a boundary where change is expected: payment vendors, databases, message channels, authentication providers, device types, report formats, or external services. Core business logic depends on the contract; adapters depend on vendors.



6.2 Seven project blueprints

Project	Contract	Concrete implementations	Benefit
Payment gateway	authenticate, pay, refund, receipt	Card, wallet, bank	Vendor replacement; consistent checkout.
Authentication framework	authenticate, revoke	Password, OAuth, passkey	Login workflow does not know provider details.
Notification framework	validate, send	Email, SMS, push	New channels do not change campaign logic.
Vehicle system	start, move, stop	Car, bicycle, drone	Simulation controls different vehicles uniformly.
Banking system	calculate interest, withdraw	Savings, checking	Rules remain specialized but account workflow stays stable.
E-commerce	quote, reserve, ship	Postal, courier, pickup	Order service depends on shipping capability, not vendor.
Hospital management	schedule, cancel, notify	In-person, telehealth, lab visit	Appointment workflow supports new visit types.

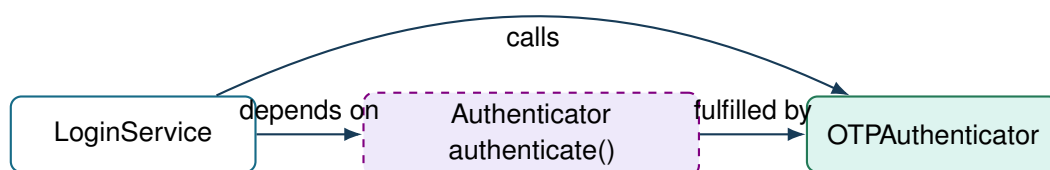
6.3 Complete mini-project: authentication framework

Problem → Pain → Need

Problem. A dashboard supports password and one-time-code login. Business code should not contain a growing `if provider == ... chain`.

Need. Depend on one authentication contract and inject the chosen provider.

Object interaction diagram



Pseudo code

```

DEFINE Authenticator contract
IMPLEMENT PasswordAuthenticator and OTPAuthenticator
INJECT one authenticator into LoginService
ASK the contract to authenticate
IF accepted, open session; OTHERWISE reject
  
```

Python implementation and driver

```

1 from abc import ABC, abstractmethod
2
3 class Authenticator(ABC):
  
```

```

4      @abstractmethod
5      def authenticate(self, user, credential):
6          """Return True only when the credential is accepted."""
7
8      class PasswordAuthenticator(Authenticator):
9          def __init__(self, password_by_user):
10             self._password_by_user = password_by_user
11
12             def authenticate(self, user, credential):
13                 return self._password_by_user.get(user) == credential
14
15      class OTPAuthenticator(Authenticator):
16          def __init__(self, code_by_user):
17             self._code_by_user = code_by_user
18
19             def authenticate(self, user, credential):
20                 return self._code_by_user.pop(user, None) == credential
21
22      class LoginService:
23          def __init__(self, authenticator):
24             self._authenticator = authenticator
25
26             def login(self, user, credential):
27                 if not self._authenticator.authenticate(user, credential):
28                     print("Access denied")
29                     return
30                 print(f"Session opened for {user}")
31
32      # Driver code
33      service = LoginService(OTPAuthenticator({"sam": "7314"}))
34      service.login("sam", "7314")
35      service.login("sam", "7314")

```

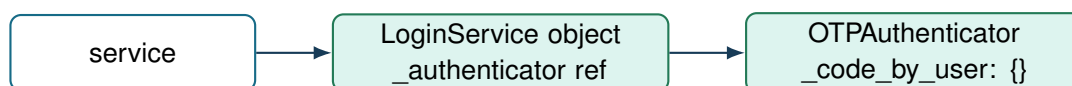
```

Session opened for sam
Access denied

```

Output explanation and dry run. The first lookup removes and returns 7314, so comparison succeeds and a session opens. The second lookup finds no code because OTPs are single-use; it compares None and denies access. Average dictionary access is $O(1)$, and storage is $O(u)$ for u users.

Memory diagram



Professional edge cases. Never store plaintext passwords; use a slow password hashing scheme. Protect OTPs, use constant-time comparison where appropriate, rate-limit attempts, expire codes, audit safely, and avoid revealing whether a user exists.

Interview Thinking

The project uses dependency inversion: high-level LoginService talks to a behavioral contract rather than a concrete provider. This improves substitution and testing. An ABC does not automatically create good architecture; the dependency direction does.

6.4 When abstraction improves maintainability

- A family of implementations shares genuinely required behavior.
- Client code should remain unchanged when a provider is added or replaced.
- The cost of an incomplete implementation is high.
- A framework is designed for other developers to extend.
- A test double should stand in for an external dependency.

6.5 When not to use it

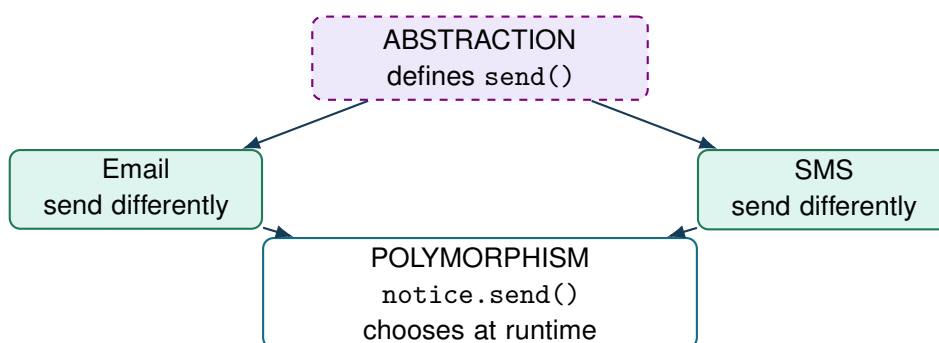
Avoid an ABC when there is one tiny implementation with no credible variation, when the common behavior is still unknown, when a simple function is clearer, or when unrelated classes are forced into an artificial hierarchy. Premature contracts freeze guesses. Start concrete, observe stable similarities, then extract an abstraction.

Separate but Connected Ideas

7.1 Abstraction versus encapsulation

Dimension	Abstraction	Encapsulation
Definition	Models essential behavior through a stable contract.	Bundles data and behavior and controls how state is accessed or changed.
Purpose	Enable safe substitution and require capabilities.	Preserve invariants and reduce accidental coupling to internals.
Focus	“What must this family do?”	“How may this object’s state be used?”
Python tools	ABCs, abstract methods, protocols, public APIs.	Classes, properties, methods, underscores, name mangling.
Payment example	Every gateway must implement pay.	Credentials remain internal and are validated before change.
Can coexist?	Yes. A good gateway often uses both.	Yes. Encapsulation supports, but does not create, the contract.
Common confusion	“Hiding details” is emphasized too strongly.	Privacy is mistaken for abstraction.
Interview sentence	Defines the essential capability.	Protects and organizes the implementation state.

7.2 Abstraction versus polymorphism



Abstraction defines the common promise. Polymorphism is the runtime ability to use that promise while

different objects respond differently. The first makes the family legible; the second makes the family useful.

7.3 Python ABCs, Java-style interfaces, protocols, and duck typing

Approach	Relationship	Enforcement/checking	Best fit
Python ABC	Explicit inheritance (usually)	Runtime instantiation check; type checkers also understand it	Shared code/state plus a nominal contract.
Java interface	A class explicitly implements an interface	Compiler verifies declared methods/types	Statically typed, explicit API contracts; modern interfaces can also have default/static methods.
Python Protocol	Structural: matching members is enough	Primarily static type checking; optional runtime checks are limited	Decoupled typing without forcing inheritance.
Duck typing	No declaration; use an object that behaves correctly	Failure when an unavailable operation executes	Small, dynamic code where flexibility outweighs formal enforcement.

★ Professional Tip

Choose an ABC when runtime category/enforcement or shared implementation matters. Choose a protocol when callers care about shape and third-party classes should qualify without inheriting. Use informal duck typing when the contract is tiny, local, and obvious.

Forty-Five Beginner Traps

The mistakes below are grouped so you can diagnose the cause, not merely memorize a fix.

#	Mistake	Why it happens / consequence	Professional correction
1	Forgetting <code>@abstractmethod</code> .	The method becomes ordinary and need not be overridden.	Mark only genuine obligations and test incomplete subclasses.
2	Forgetting to inherit ABC.	The marker may not be enforced through the intended setup.	Inherit ABC (or deliberately use <code>ABCMeta</code>).
3	Instantiating the ABC.	Confusing blueprint with finished product.	Instantiate a concrete subclass.
4	Implementing only some obligations.	One missing method keeps the child abstract.	Read the <code>TypeError</code> ; implement every listed method.
5	Misspelling a method name.	A new method is created; the old obligation remains.	Use type checking, IDE completion, and tests.
6	Wrong capitalization.	Python names are case-sensitive.	Match the contract exactly.
7	Treating pass as implementation.	Instantiation succeeds but behavior is fake.	Implement real semantics; test results, not merely names.
8	Assuming docstrings enforce rules.	Documentation informs humans only.	Use ABCs/tests where enforcement matters.
9	Using <code>NotImplementedError</code> as an ABC substitute.	Failure is late and call-dependent.	Use <code>@abstractmethod</code> for required overrides.
10	Believing abstract methods cannot have code.	“Abstract” is confused with “empty.”	Use a body when shared partial behavior is useful.
11	Assuming parent abstract code runs automatically.	Overrides replace lookup behavior.	Call <code>super()</code> explicitly if desired.
12	Reversing decorator order.	Descriptors may reject changing the abstract flag.	Put <code>@abstractmethod</code> nearest <code>def</code> .
13	Omitting <code>self</code> .	Instance method calling convention is broken.	Match an instance-method signature.
14	Incompatible parameters.	Python ABC runtime checks names, not full substitutability.	Keep compatible signatures; use a type checker/tests.
15	Returning a different meaning.	Same name falsely suggests substitutability.	Document and preserve semantic contracts.

#	Mistake	Why it happens / consequence	Professional correction
16	Changing exception behavior wildly.	Callers cannot rely on the abstraction.	Document allowed exceptions and translate vendor errors.
17	Making every method abstract.	Shared behavior is duplicated.	Keep stable algorithms concrete.
18	Making nothing abstract.	The “ABC” communicates no enforceable obligation.	Either add meaningful requirements or use a normal class.
19	Creating an ABC for one trivial class.	Ceremony exceeds benefit.	Start concrete until variation is credible.
20	Predicting future abstractions too early.	The contract encodes guesses.	Extract after observing stable common behavior.
21	Huge “god” ABC.	Implementers receive irrelevant obligations.	Split into focused capability contracts.
22	Forcing unrelated classes into inheritance.	Superficial similarities hide different semantics.	Prefer composition or protocols.
23	Confusing abstraction with encapsulation.	Both are described as hiding.	Ask “required behavior or controlled state?”
24	Confusing abstraction with polymorphism.	They often appear together.	Contract defines; polymorphism dispatches.
25	Thinking ABC equals Java interface.	Language models differ.	Remember Python ABCs can hold state and implementations.
26	Ignoring duck typing.	ABCs are treated as the only design choice.	Choose formality proportional to risk.
27	Ignoring protocols.	Explicit inheritance creates avoidable coupling.	Use structural typing when shape alone matters.
28	Checking <code>type(x) is Child</code> .	Valid subtypes are rejected.	Depend on behavior; if needed use <code>isinstance</code> .
29	Using <code>isinstance</code> everywhere.	Client code becomes a type-switch.	Call the polymorphic operation directly.
30	Putting vendor details in the ABC.	The stable boundary becomes unstable.	Express domain capabilities, not one adapter’s API.

#	Mistake	Why it happens / consequence	Professional correction
31	Leaking secrets through shared fields.	Reuse is mistaken for safe storage.	Encapsulate credentials; follow security practices.
32	Calling overridable methods in <code>__init__</code> .	Child state may not yet exist.	Finish construction before polymorphic calls.
33	Forgetting <code>super().__init__</code> .	Shared state is absent.	Call it when the base constructor establishes invariants.
34	Assuming ABC validates business correctness.	It checks presence, not quality.	Add tests, validation, monitoring, and review.
35	Assuming annotations are runtime-enforced.	Type hints are usually not runtime guards.	Run a static type checker and validate external data.
36	Swallowing exceptions in implementations.	Contract appears successful after failure.	Define failure semantics and surface errors responsibly.
37	Inconsistent side effects.	Equivalent calls behave unpredictably.	Document idempotency and state changes.
38	No tests for each implementation.	One adapter honors syntax but violates meaning.	Run a shared contract-test suite against all adapters.
39	Testing only the ABC.	Most risk lies in concrete integrations.	Test unit fakes and integration adapters.
40	Modifying the ABC casually.	Every child may break.	Version public contracts and evolve deliberately.
41	Removing an abstract method without design review.	Invariants may silently weaken.	Treat contract changes as API changes.
42	Overusing multiple inheritance of ABCs.	Method resolution becomes hard to reason about.	Prefer small capability ABCs and composition.
43	Ignoring cooperative <code>super()</code> .	Diamond hierarchies can skip or repeat work.	Design compatible signatures or avoid the hierarchy.
44	Returning <code>NotImplemented</code> .	It is confused with raising <code>NotImplementedError</code> .	Reserve <code>NotImplemented</code> for binary operator fallback.
45	Calling a convention a guarantee.	A leading underscore is not access control.	Distinguish social convention from runtime enforcement.

Interview Thinking

9.1 Questions with reasoning-first answers

Interview Thinking

Why does abstraction exist?

Systems contain families of components that must offer stable capabilities while their implementations vary. Abstraction lets callers depend on those capabilities and, with an ABC, rejects incomplete concrete subclasses early.

Interview Thinking

Why is normal inheritance insufficient?

Inheritance supplies reuse and method lookup. An ordinary parent method may be inherited unchanged; it does not force overriding. `@abstractmethod` turns a selected method into an outstanding obligation that blocks instantiation.

Interview Thinking

Can an abstract class have a constructor, fields, and concrete methods?

Yes. “Abstract” means the class has at least one unfulfilled abstract obligation, not that the class contains no implementation. Shared state and algorithms are common reasons to choose an ABC rather than a purely structural protocol.

Interview Thinking

Can an abstract method contain code?

Yes. A child must still override it, and may call the shared parent body using `super()`. Prefer a separate concrete helper when that communicates the reuse more clearly.

Interview Thinking

Why use a contract instead of documentation?

Documentation explains intent but cannot reject an incomplete object. An ABC provides an executable check. It still cannot prove correct semantics, so documentation and tests remain necessary.

Interview Thinking**When should abstraction not be used?**

When there is no stable family, no meaningful substitution, a simple function or concrete class is clearer, or the hierarchy forces unrelated implementations to pretend they share semantics.

9.2 A strong 60-second explanation

“Suppose every payment provider must authenticate and pay, but each provider uses a different algorithm. Ordinary inheritance lets a developer forget one method and still create the child object. An abstract base class declares those operations as a behavioral contract. Python prevents instantiation until a concrete subclass implements all abstract methods. The contract defines *what*; overriding supplies different *hows*; polymorphism lets client code use them uniformly. Encapsulation is different: it controls access to internal state. I use ABCs at stable, high-value extension boundaries, not for every small class.”

Practice Studio

10.1 Easy: build the mental model

Practice: Concept checks

1. In one sentence, explain the pain that abstract methods solve.
2. Why is `pass` in a required ordinary parent method dangerous?
3. Label each as WHAT or HOW: `Shape.area`; Heron's formula; rectangle width times height.
4. Can an abstract class be used in a type annotation even though it cannot be instantiated?
5. Write two required behaviors for a `ReportExporter` contract.

10.2 Medium: implementation

Practice: Shape contract

Create an ABC `Shape` requiring `area()` and `perimeter()`. Implement `Rectangle` and `Circle`. Add a concrete `describe()` method. Include driver code and predict output before running. Test that an incomplete `Triangle` cannot be instantiated.

Practice: Employee payroll

Create `Employee.calculate_pay()` as an abstract method. Implement `salaried` and `hourly` employees. Write `print_payroll(employees)` that uses polymorphism. State the time complexity for n employees.

10.3 Hard: design

Practice: E-commerce shipping

Design small contracts for quoting, tracking, and cancellation. Not every pickup option supports tracking. Decide whether one large ABC or several capability ABCs is more honest. Draw the relationship diagram and defend your decision.

Practice: Hospital appointments

Model in-person, telehealth, and laboratory appointments. Require scheduling and cancellation; decide where patient notification belongs. Include failure semantics, shared contract tests, and a fake implementation for unit tests.

10.4 Debugging challenges

Practice: Find the contract bug

Predict the result, then repair it.

```
1 from abc import ABC, abstractmethod
2 class Vehicle(ABC):
3     @abstractmethod
4     def start(self): ...
5 class Scooter(Vehicle):
6     def Start(self): print("Ready")
7 s = Scooter()
```

Practice: Fix decorator order

Explain why this design is wrong and correct it.

```
1 class Document(ABC):
2     @abstractmethod
3     @property
4     def title(self): ...
```

10.5 Output prediction

Practice: Shared abstract body

Predict every line.

```
1 class A(ABC):
2     @abstractmethod
3     def run(self): print("A")
4 class B(A):
5     def run(self):
6         print("B1")
7         super().run()
8         print("B2")
9 B().run()
```

10.6 Fill in the blanks

Practice: Complete the notification contract

```
1 from abc import _____, _____
2 class Notification(_____):
3     _____
4     def send(self, message): ...
5 class SMS(Notification):
6     def _____(self, message):
7         print(message)
```

10.7 Interview coding tasks

1. Build a cloud storage ABC and two in-memory test implementations with different capacity policies.
2. Refactor an if/elif payment type-switch into injected polymorphic gateways.
3. Create a shared contract-test function that accepts a storage factory and verifies upload/download/delete semantics.
4. Show an abstract property and explain decorator order.
5. Demonstrate an abstract method with reusable parent code, then refactor it into a concrete helper; compare clarity.

10.8 Selected answers

Check your reasoning

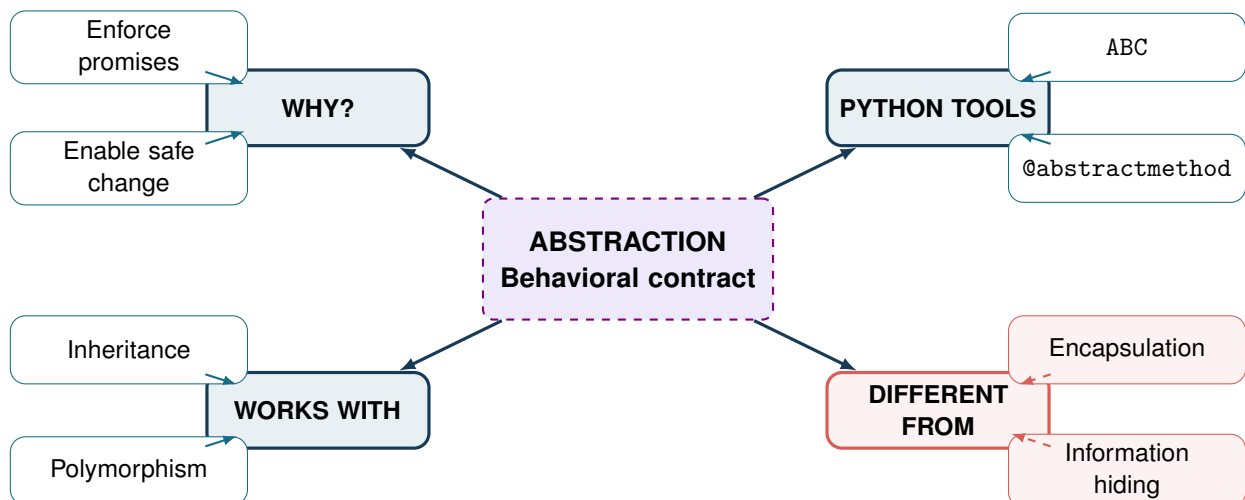
Easy 3: area is WHAT; Heron's formula and width-times-height are HOW. Debugging 1: Start does not fulfill start, so instantiation raises TypeError. Debugging 2: @property must be outermost and @abstractmethod closest to def. Output prediction: B1, A, B2. Fill-ins: ABC, abstractmethod, ABC, @abstractmethod, send.

Master Revision Map

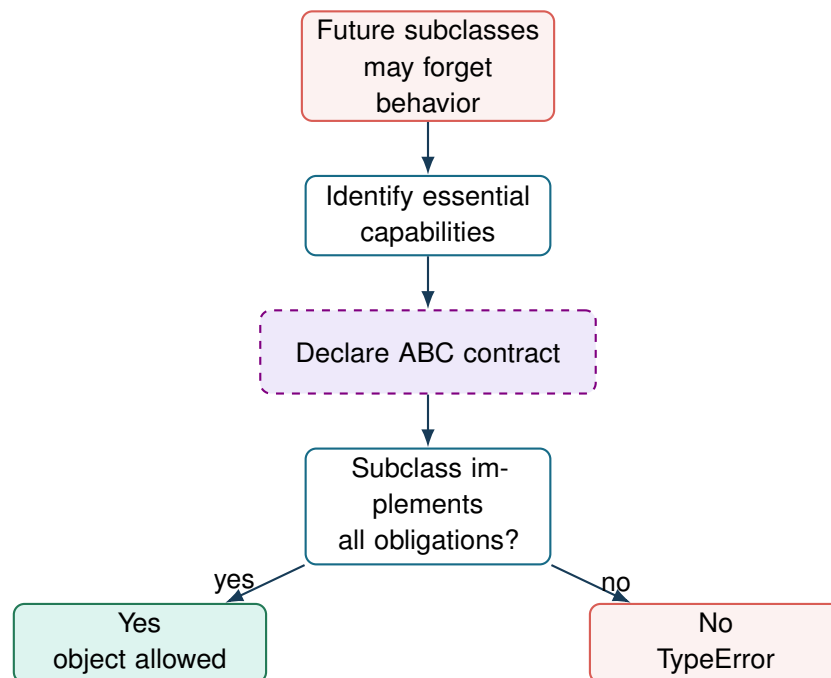
11.1 Cheat sheet

Core syntax	Core logic
<pre> from abc import ABC, abstractmethod class Contract(ABC): @abstractmethod def required(self): ... def shared(self): print("Reusable") </pre>	• ABC = incomplete behavioral blueprint. • Abstract method = required override. • Concrete child = all obligations fulfilled. • Abstract class object = blocked. • Concrete methods/state = allowed. • Presence is checked; correctness is tested.

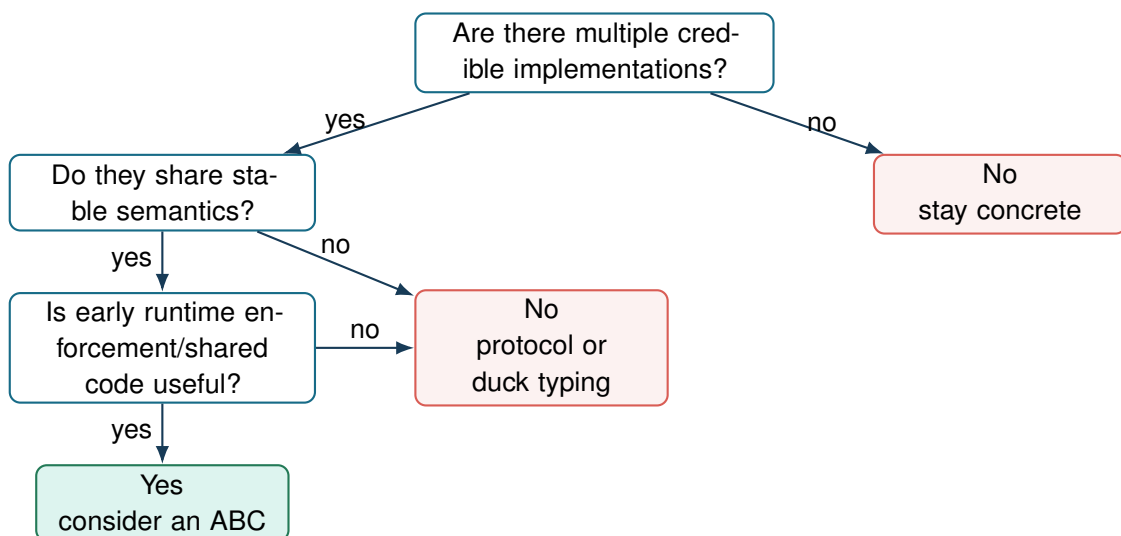
11.2 Mind map



11.3 Flow chart: from pain to working object



11.4 Decision tree: should I create an abstraction?



11.5 Final revision notes

1. Begin with the pain: ordinary inheritance permits forgotten overrides.
2. A contract lists capabilities every concrete member must honestly provide.
3. Abstraction defines WHAT; subclasses supply HOW.
4. ABC and @abstractmethod make selected promises executable.
5. Incomplete subclasses may exist as intermediate designs but cannot produce objects.
6. Abstract classes can contain constructors, data, concrete methods, and abstract members.
7. Abstract methods may contain reusable code; children still override them.
8. Polymorphism uses the contract; encapsulation protects implementation state.

9. Protocols and duck typing are alternatives, not enemies.
10. Extract abstractions at stable boundaries; do not manufacture hierarchies for ceremony.

The one idea to keep

Abstraction is not a magic way to hide code. It is a design conversation made executable: “Any object claiming to belong to this family must be able to do these essential things. How it does them is its own responsibility.”

Instructor's Exit Check

You are ready when you can look at an incomplete future subclass and explain, without reciting a definition, (1) the damage it could cause, (2) the behavioral promise that would prevent it, (3) why ordinary inheritance cannot enforce that promise, and (4) how Python's ABC machinery rejects the incomplete object while preserving different implementations.